

Smart Warehouse Management System

Version 1.0 | C++ Object-Oriented Programming & FLTK Training Module

Prepared for: Software Engineering Trainees & Development Team

Language: C++ (Classes, STL Containers, FLTK GUI)

INTERNAL TRAINING DOCUMENT - NOT FOR PUBLIC DISTRIBUTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and design requirements for implementing the Smart Warehouse Management System in C++. It is structured to guide team members through developing and integrating their respective modules into a cohesive, object-oriented desktop application.

By the end of this project, the team will demonstrate mastery of:

- Designing collaborative C++ classes with clear separation of concerns and modular encapsulation.
- Managing collections of objects using Standard Template Library (STL) containers (`std::vector`).
- Implementing constructors, accessors, mutators, and input validation logic.
- Developing a lightweight native graphical user interface using **FLTK (Fast Light Toolkit)**.
- Writing clean, correct object-oriented code without memory leaks or index errors.

1.2 Project Overview

The system is a standalone, desktop-based C++ application that simulates a smart warehouse management environment. It is composed of six related classes assigned across the team: User, Product, Category, Supplier, Order, and GUIController. All data exists in memory during runtime to facilitate rapid iteration and testing.

1.3 Scope

The implementation covers the following six core classes:

- **User:** Stores account credentials (username, password, phone), operational status, and session tracking.
- **Product:** Stores inventory item metadata (ID, name, quantity, unit price).

- **Category:** Manages item classification groupings.
- **Supplier:** Tracks vendor contact details and supply linkages.
- **Order:** Records inbound and outbound transaction workflows.
- **GUIController:** Manages native FLTK windows, layout containers, tabbed panes, and interactive data tables.

1.4 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
User	A person operating the warehouse system; represented by the User class.
Product	An individual inventory item tracked by the system.
Supplier	A vendor providing goods to the warehouse.
FLTK	Fast Light Toolkit; a cross-platform C++ GUI library.

2. System Design Overview

2.1 Class Architecture

The application follows a structured, multi-module paradigm where data models (User, Product, Supplier, Order) are managed cleanly and passed into the GUIController layer for rendering. Standard STL vectors are utilized to hold runtime datasets securely in memory.

2.2 Design Conventions

- **Target Platform:** Windows 10/11.
- **C++ Standard:** C++17.
- **Interface:** Graphical Desktop Interface via FLTK.
- **Dependencies:** FLTK 1.4+ and standard C++ libraries.

3. Class Specifications

3.1 User Class (Team Member 1)

- **Data Members:** string username, string password, string phone, string status, string lastSeen.

- **Constructor Requirements:** Default and parameterized constructors to initialize credentials and timestamps.
- **Method Requirements:**
 - getUsername(), getPhone(), getStatus(), getLastSeen() (Read accessors).
 - setStatus(), setPhone(), updateLastSeen() (Mutators).
 - checkPassword(), changePassword() (Security handlers).

3.2 Product & Category Classes (Team Member 2)

- **Product Data Members:** int id, char name[64], int quantity, float price, string category.
- **Category Data Members:** int categoryId, char name[32].
- **Method Requirements:** Support CRUD operations, stock level updates, and catalog lookups.

3.3 Supplier Class (Team Member 3)

- **Data Members:** int supplierId, char name[64], char contactEmail[64], char suppliedProduct[32].
- **Method Requirements:** Manage vendor profiles, contact updates, and linked product association.

3.4 Order Class (Team Member 4)

- **Data Members:** int orderId, int productId, int quantityOrdered, char orderType[16] ("Inbound" or "Outbound").
- **Method Requirements:** Process transactions, validate stock availability, and log order history.

3.5 GUIController Class (Team Member 5 & Yassin)

- **Responsibilities:** Construct FLTK windows, layout buttons, text input fields, and tabbed dashboard views.
- **Method Requirements:** Render live inventory grids, supplier lists, and transaction forms with real-time UI response updates.

4. Edge Cases & Error Handling

- **Duplicate Usernames:** System must reject sign-up requests if a username already exists.
- **Password Constraints:** Passwords must meet minimum length criteria before registration is accepted.
- **Invalid Input:** Non-numeric entries in quantity or price fields must be caught gracefully without crashing the UI.
- **Stock Underflow:** Outbound orders exceeding current stock levels must trigger an explanatory warning message.

5. Functional Requirements (FR)

- **FR1:** Users must register with a unique username, secure password, and contact phone.
- **FR2:** The inventory module must support adding, updating, and viewing product catalog details.
- **FR3:** The supplier module must track vendor contact information and associated supply items.
- **FR4:** The order module must record transaction histories and update inventory stock counts.
- **FR5:** The FLTK interface must display system metrics, tabs, and data tables clearly.

6. Implementation Checklist

- ☐ **Phase 1:** User class authentication & profile management.
- ☐ **Phase 2:** Product and Category inventory structures implemented.
- ☐ **Phase 3:** Supplier and Order transaction logic finalized.
- ☐ **Phase 4:** FLTK GUIController views and form callbacks integrated.
- ☐ **Phase 5:** Build verification via MSYS2 UCRT64 (g++ -std=c++17).